

Reflect-SQL: A Self-Reflection Based Framework for Text-to-SQL

Anupreksha Jain¹[0009–0004–5609–2108] and Manish Shrivastava¹[0000–0001–8705–6637]

International Institute of Information Technology Hyderabad, India
anupreksha.j@research.iiit.ac.in
m.shrivastava@iiit.ac.in

Abstract. Democratizing data access through natural language is a crucial goal for modern enterprises, but the practical adoption of Text-to-SQL is critically hindered by real-world complexities: 1. Obscure and large database schemas, 2. Ineffective retrieval of relevant tables and columns due to structured setting of schemas and vague user query, 3. Generation of syntactically or logically flawed SQL due to a lack of robust validation and correction mechanism. To address these systemic challenges, we introduce Reflect-SQL, a novel framework for Text to SQL, grounded in multi-stage self-reflection approach to develop understanding of obscure schema using a knowledge base, setup a process for effective retrieval and system to generate syntactically/semantically SQL. Instead of a single-pass attempt, our system employs an LLM-as-a-judge driven scoring mechanism within interconnected feedback loops to iteratively refine the results at every stage. A feedback-driven retrieval loop refines the user’s natural language query, while a synthesis loop validates and corrects the SQL and finally, an entailment loop optimizes the end-to-end process and continuously enriches the knowledge base. By integrating these layers of reflection, Reflect-SQL bridges the critical gap between user intent and complex data. On the challenging BIRD benchmark, our framework achieves an execution accuracy of 72.03%, significantly outperforming state-of-the-art baselines, demonstrating a major leap in reliability for enterprise applications.

Keywords: Text-to-SQL · Self-reflection · Retrieval-Augmented Generation · Knowledge Base · Large Language Models.

1 Introduction

Successfully deploying Text-to-SQL in enterprise environments hinges on overcoming three critical challenges:

1. Schema Obscurity: Deciphering large-scale databases characterized by cryptic column names (e.g., 'A2', 'dname') and undocumented relationships.
2. Contextual Retrieval: Accurately extracting relevant schema elements is difficult, as vague user queries often fail to map clearly onto complex, structured database environments.

3. **Logical Integrity:** Moving beyond syntax to ensure semantic accuracy, preventing the generation of logically flawed SQL that yields incorrect results.

We introduce **Reflect-SQL**, a framework that overcomes these challenges through multi-stage self-reflection. The system utilizes a dynamic **Knowledge Base** to disambiguate cryptic database structures, enabling a **Feedback-Driven Retrieval** process that refines vague user queries for precise context. Additionally, an **Iterative SQL Synthesis** loop validates and corrects the generated code, using a final entailment check to guarantee that the results match the user’s original intent.

Our experiments validate this multi-layered, reflective approach, demonstrating a significant improvement in the accuracy of 11.05% on complex benchmarks. The primary contributions of this work are:

1. **A complete Text-to-SQL Framework:** A comprehensive, end-to-end system for enterprise Text-to-SQL tasks.
2. **Dynamic Knowledge Base:** A learning mechanism that enriches schema understanding over time.
3. **Reference-Free SQL Evaluation:** A novel methodology for validating SQL without relying on gold-standard labels.
4. **Feedback-Driven Query Refinement:** An iterative process that aligns ambiguous user queries with the database schema.

2 Related work

[1] introduced the concept of self-reflection in language models to progressively improve the factuality and consistency of generated responses, primarily for open-domain tasks. In their approach, the model scores its own output and iteratively refines it on the basis of identified errors. Our work adapts and extends this paradigm to the structured domain of Text-to-SQL generation—a task with fundamentally different constraints and evaluation criteria for Knowledge Base Construction, Information Retrieval and SQL Generation.

2.1 Knowledge Base Construction

Recent work has established that rich knowledge bases, built by aligning meta-data with business glossaries [2], enhancing column descriptions [3], [4], [5], or injecting domain knowledge [6], significantly improve Text-to-SQL performance. However, these methods primarily treat the knowledge base as a static asset created once. Our approach introduces an iterative refinement loop, where the knowledge base is continuously enriched based on successful query executions, allowing it to adapt to ambiguous, real-world applications.

2.2 Information Retrieval

Retrieval-Augmented Generation (RAG), introduced by [7] has evolved to include iterative query refinement to improve context quality [8], [9], [10]. We

adapt this principle to the structured domain of databases by creating a schema aware, hierarchical retrieval process. Unlike standard RAG, our framework retrieves information iteratively from Knowledge base (tables, related tables, and columns) and uses a concrete feedback score to reformulate the user’s natural language query if the retrieved context is insufficient or irrelevant.

2.3 Text-to-SQL

The Text-to-SQL field has evolved from early encoder-decoder architectures like Seq2SQL [11] and SQLNet [12] to the current paradigm dominated by Large Language Models (LLMs). Modern in-context learning methods, such as DIN-SQL [13] and SQLPrompt [14], have significantly improved performance leveraging few-shot exemplars and chain-of-thought reasoning. Our framework not only builds on these foundations but also introduces critical layers of self-correction. First, we employ an iterative SQL synthesis loop that goes beyond syntax to correct logical errors based on a multi-dimensional semantic score. Second, as a final safeguard, we add an entailment check that validates if the query’s results actually answer the user’s intent.

3 Knowledge base construction and refinement

The knowledge base (KB) acts as a semantic abstraction layer containing table descriptions, column metadata, and relationships. This component is vital for handling industry-scale databases, which often exceed the context limits of Large Language Models (LLMs) and suffer from obscure naming conventions. By externalizing schema details into a structured KB, we overcome context constraints and limited schema understanding, ensuring the LLM accesses only the specific information required to bridge the gap between user intent and the database. The overall architecture of this framework is depicted in Fig. 1.

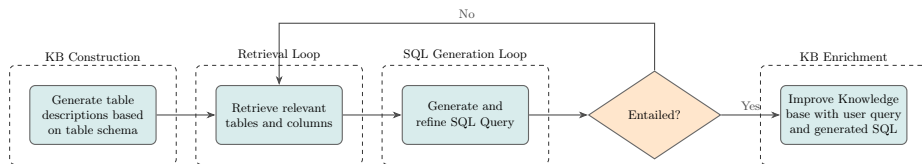


Fig. 1: High-level architecture of the Reflect-SQL framework.

3.1 Construction

The knowledge base serves as the semantic backbone of our framework. For the BIRD[15] dataset, we construct this repository by synthesizing the database

schema, sample data values, and the provided training set of sample questions and external evidence. In broader enterprise settings, this process also ingests domain-specific knowledge from business documents and glossaries. These inputs are processed to generate comprehensive descriptions for tables, columns, and their relationships (a detailed JSON representation of this structure is provided in Appendix B). To enable efficient retrieval, these components are encoded into dense vector representations using a sentence embedding model [16]. The vector database is structured hierarchically: a global collection for table descriptions and separate local collections for each table’s column descriptions. This architecture supports a two-step retrieval process, first identifying relevant tables and subsequently narrowing down to specific columns.

3.2 Iterative refinement

In this post-processing step, after a successful SQL execution, if the query passes syntactic and semantic validations and entailment score between the query and result exceeds a threshold θ_e , the knowledge base is refined. This conditional update is critical for preventing the propagation of errors. low-quality or contextually incorrect queries might pollute the knowledge base. Only syntactically and semantically validated queries with validated results contribute to the system’s learning process.

This iterative update mechanism is essential in real-world environments, particularly those characterized by ambiguous or indecipherable column names and continuously evolving schemas. Example in Table 1 shows how the description of columns are refined after a successful query execution.

Component	Before Refinement	After Refinement
loan.status	Categorical column for loan status with values 'A', 'B', 'C', 'D'.	Loan repayment state: 'A' indicates finished or paid, 'B' indicates defaulted...
trans.k_symbol	Text column named 'k_symbol' with values 'POJISTNE', 'SIPO'.	Transaction Category: determines the payment type (e.g. 'POJISTNE' maps to Insurance)...

Table 1: Example of Knowledge Base Refinement for BIRD[15]

4 Iterative RAG retrieval

In the initial retrieval phase, we use the curated KB to identify relevant tables and columns for a user query, as detailed in Fig. 2.

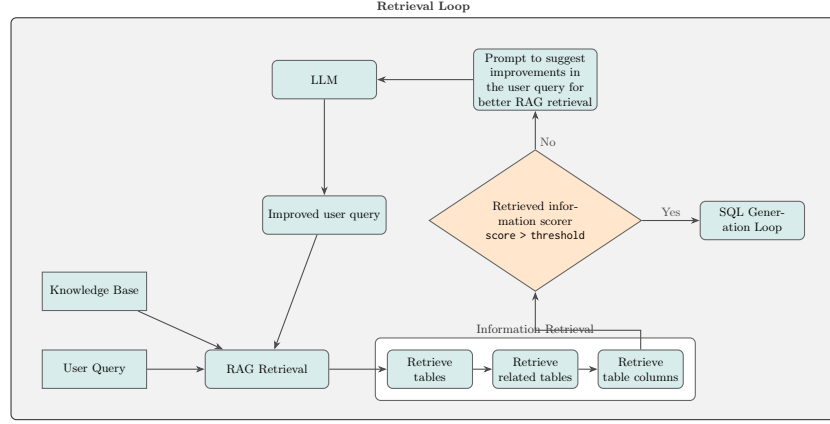


Fig. 2: Iterative Retrieval Loop.

4.1 Table and column retrieval (hierarchical retrieval process)

Given a user query Q , we embed it and use cosine similarity(as shown in 1) to retrieve the top-k tables $\{T_1, \dots, T_k\}$ from the vector database.

$$\text{score}(T_i, Q) = \cos(\mathbf{t}_i, \mathbf{q}) = \frac{\mathbf{t}_i \cdot \mathbf{q}}{\|\mathbf{t}_i\| \|\mathbf{q}\|} \quad (1)$$

This set is expanded by including tables related via relationships defined in the KB, ensuring join paths are available. For each selected table, a similar embedding-based search retrieves the most relevant columns.

4.2 Relevance and completeness scoring

We compute two metrics to assess retrieval quality: a **Relevance Score** (R) for semantic alignment and a **Completeness Score** (C) to ensure all necessary schema elements are present. These scores are generated by a novel approach of LLM-as-a-judge scoring (prompt details are added in appendix A). Final retrieval score ($Score_r$) is calculated with formula 2.

$$Score_r = \alpha R + (1 - \alpha)C, \quad \text{where} \quad \alpha \in [0, 1] \quad (2)$$

4.3 Query Refinement

If the retrieved schema is deemed insufficient ($Score_r < \theta_r$), an iterative refinement loop is triggered. In each iteration, the LLM first reformulates the user query into a more detailed version, Q' , to improve its semantic focus. Simultaneously, the retrieval scope is expanded by increasing the number of tables and columns to be retrieved in the next pass. This process of refining the query's focus while broadening the search area allows the system to quickly zero in on

the complete set of schema elements required for complex queries. The loop terminates once the $Score_r$ exceeds the threshold θ_r or a maximum number of iterations is reached.

5 SQL query generation

With a high-quality set of retrieved tables, columns, descriptions and sample data we generate a candidate SQL query using an LLM, as shown in Fig. 3.

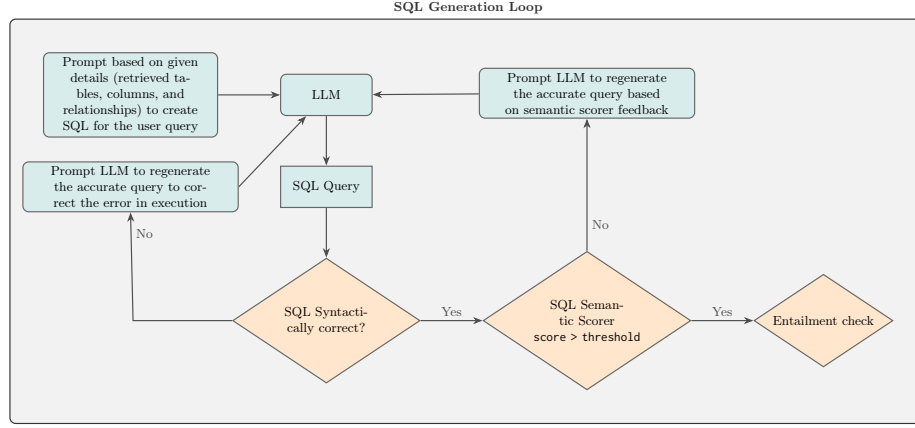


Fig. 3: Iterative SQL Generation Loop.

5.1 Syntactic Validation and Correction Loop

Each generated SQL query is immediately subjected to a syntactic validation check using a database engine. In the event of a syntax error, a correction loop is triggered. The system feeds the original query, the resulting error message, and the schema context back to the LLM with instructions to fix the error. This loop repeats until a syntactically correct query is produced or a maximum iteration limit is met, making the system resilient to basic generation mistakes.

5.2 Semantic validation loop

A syntactically valid query undergoes semantic validation across four dimensions: Column name consistency (C), Granularity and aggregation (G), Join and relationship consistency (J), and Query logic alignment (Q). An LLM-as-a-judge (prompt in Appendix A) scores each dimension, and a final semantic score S is calculated as a weighted sum of scores as shown in 3.

$$S = w_c \cdot C + w_g \cdot G + w_j \cdot J + w_q \cdot Q \quad (3)$$

where w_c, w_g, w_j, w_q are predefined weights. If S is below threshold θ_s , the query is sent back to the LLM with diagnostic feedback for iterative correction.

6 Entailment

Once a query passes all validation, it is executed in database engine to retrieve the results. In a final verification stage, we check if the result set R logically entails the user’s original query Q .

6.1 Entailment scoring

As shown in 4 an entailment score $E(Q, R)$ is computed by an LLM-as-a-judge (see Appendix A), assessing both **Schema consistency** (E_{schema}) and **Data consistency** (E_{data}).

$$E(Q, R) = \beta \cdot E_{\text{schema}} + (1 - \beta) \cdot E_{\text{data}} \quad \text{where} \quad \beta \in [0, 1] \quad (4)$$

6.2 Entailment decision

If $E(Q, R) \geq \theta_e$, the result is returned to the user, and the KB is updated with the successful query-result pair. If not, the entire pipeline is re-triggered with feedback on the failure reason (e.g., missing columns, incorrect filters). This final loop ensures the system’s output truly matches the user’s intent.

7 Mitigating Bias in LLM-based Evaluation

We acknowledge that using an LLM as a judge can introduce potential biases [17]. To mitigate this, our framework employs a structured, few-shot prompting strategy for all scoring tasks. As shown in Appendix A, our prompts provide detailed instructions, scoring rubrics, and few-shot examples. These examples anchor the LLM’s judgment to our specific, task-aligned criteria, increasing the reliability and reproducibility of the evaluation while reducing the impact of the model’s inherent biases.

8 Model Parameter Optimization

We conducted a systematic grid search to determine the optimal values for all parameters: thresholds ($\theta_r, \theta_s, \theta_e$) and weights ($\alpha, \beta, w_c, w_g, w_j, w_q$). Each parameter was varied and the combination that yielded the highest accuracy in our validation set was selected. The final optimized values are presented in Table 2.

Table 2: Optimized Parameters using grid search.

Parameter	Value
θ_r	4
θ_s	5
θ_e	3
α	0.6
β	0.2
w_c	0.1
w_g	0.3
w_j	0.4
w_q	0.2

9 Evaluation

9.1 Experimental setup

We evaluated our framework using the **BIRD benchmark** [15]. Given that accessing large-scale, real-world enterprise datasets for academic experiments is often infeasible due to their proprietary nature, the BIRD benchmark serves as an excellent proxy. It is designed to test Text-to-SQL systems over diverse relational databases that mirror real-world complexity, making it a suitable testbed for our system.

9.2 Results

We evaluated using Claude, OpenAI, DeepSeek API based, and DeepSeek open-source family models. Table 3 presents the execution accuracy on the BIRD dataset. Among all models, enabling our reflection framework resulted in substantial accuracy gains over evaluation without reflection framework with overall 6-8% gain in accuracy. These results confirm that our iterative reasoning and refinement mechanism plays a critical role in enhancing performance.

9.3 Human evaluation

To complement our automatic evaluation, we conducted a qualitative human assessment to analyze the correctness and semantic alignment of generated queries, particularly focusing on the impact of the entailment check. This evaluation centered on identifying subtle logical errors that might not be caught by syntax or basic semantic validation alone to validate how framework works. Table 4 shows one such example.

9.4 Comparison with State-of-the-Art

We benchmarked our framework against leading State-of-the-Art methods on the BIRD development set (Table 5). Reflect-SQL achieves a state-of-the-art execution accuracy of 72.03%, marking a significant improvement over DIN-SQL

Table 3: Accuracy (%) of various models on the BIRD dataset. Top performers in each category are highlighted.

Model	Simple	Moderate	Challenging	Overall
Claude Models				
Claude-sonnet-4.5 (with reflection)	88.97	50.65	32.41	72.03
Claude-sonnet-4.5 (w/o reflection)	81.18	45.25	24.13	64.92
OpenAI API Models				
o1-preview (with reflection)	75.89	48.49	18.62	62.19
o1-preview (w/o reflection)	69.35	40.54	15.86	55.66
DeepSeek API Models				
deepseek-reasoner (with reflection)	73.62	46.55	17.24	60.10
deepseek-reasoner (w/o reflection)	62.90	40.54	12.41	51.88
Open Source Models (Local setup)				
DeepSeek-Coder-V2-Lite... (with reflection)	43.55	21.62	3.44	35.02
DeepSeek-Coder-V2-Lite... (w/o reflection)	30.65	13.51	0.0	22.64

Table 4: Example of SQL refinement driven by entailment feedback.

Stage	Analysis / SQL Snippet
User Query	List the lowest three eligible free rates for students in continuation schools.
Initial SQL	SELECT ... (f.Count / f.Enrollment) ... WHERE s.SOCType LIKE '%Continuation%'
Entailment Check Result	Score: 2 / 5 Reason: The result returns None values due to division by zero or nulls. Filters for positive enrollment and float casting are needed.
Refined SQL	SELECT ... (CAST(f.Count AS REAL) / f.Enrollment) ... WHERE f.Enrollment > 0 AND s.SOCType LIKE '%Continuation%'

[13] (50.72%), DAIL-SQL (57.41%) and MAC-SQL (59.59%). This performance is comparable to the recently proposed CHASE-SQL architecture [21], establishing Reflect-SQL as a top-tier solution for complex database querying.

10 Ablation Study

To understand the contribution of each component, we performed an ablation study using deepseek-chat as the base model. As shown in Table 6, removing any component of the Reflect-SQL framework degrades performance. The self-reflection loop in SQL generation proved most critical, with its removal causing the sharpest drop in accuracy. These findings affirm the effectiveness of our multi-stage, reflective approach.

Table 5: Comparison with State-of-the-Art methods on BIRD dev set (Execution Accuracy %).

Method	Overall Accuracy
BIRD paper [15]	46.35
DIN-SQL [13]	50.72
DAIL-SQL [19]	57.41
MAC-SQL [20]	59.59
CHASE-SQL + Claude 3.5 Sonnet [21]	69.53
CHASE-SQL + Gemini 1.5 Pro [21]	73.01
Reflect-SQL + Claude 4.5 Sonnet (Ours)	72.03

Table 6: Ablation results showing the contribution of each module.

Variant	Simple	Moderate	Challenging	Overall
Full Model (Self-Reflect)	68.64	43.10	14.48	55.80
w/o Self-Reflection in SQL Generation	62.91	37.93	12.41	50.58
w/o Iterative Retrieval	67.02	40.30	10.34	53.58
w/o Entailment Check	64.10	41.59	13.10	52.47

11 Conclusion and Future Work

In this work, we presented Reflect-SQL, a self-reflection framework that leverages iterative retrieval, generation, and validation to produce accurate SQL queries. Our approach addresses key challenges in large-schema environments by refining user queries and SQL outputs in a closed feedback loop. By continuously updating its knowledge base, the system adapts over time, enhancing its performance for real-world use cases. In future work, we plan to:

1. Explore graph-based schema representations to better model complex inter-table relationships and improve the efficiency of our hierarchical retrieval.
2. Develop more advanced entailment scoring methods, using techniques like contrastive learning to enhance the framework’s ability to detect subtle misalignments between results and user intent.

12 Limitations

Our framework’s effectiveness depends on the initial quality of the knowledge base and can face scalability challenges with extremely large schemas. Furthermore, the automatic evaluation metrics for semantic correctness and entailment, while effective, are still an evolving area of research.

A Example Prompts for LLM-as-a-judge Scoring

Prompt for Semantic Scoring

```
You are an expert SQL analyst. Score a generated SQL query on four
dimensions of semantic correctness (1-10).
**User Query:** "{user_query}"
**Database Schema:** {schema_json}
**Generated SQL:** "{sql_query}"
**Instructions:**
1. **Column Name Consistency:** Are all columns valid?
2. **Granularity and Aggregation:** Are aggregations/filters correct?
3. **Join and Relationship Consistency:** Are joins logical?
4. **Query Logic Alignment:** Does the logic match the user's intent?
**Few-shot Examples:**[...]
**Current Task:**
Evaluate the provided query, schema, and SQL. Respond in JSON format:{...}
```

B Knowledge Base Structure

Figure 4 illustrates the structured JSON format of the constructed Knowledge Base. This representation captures the semantic descriptions of tables and columns derived from the raw schema and auxiliary evidence, serving as the source for vector embedding generation.

```
{
  "financial": {
    "account": {
      "table_description": "it stores information about bank accounts ...",
      "relationships": [{
        "column_name": "district_id", "related_table": "district",
        "related_column": "district_id"
      }...],
      "column_description": {
        "account_id": "Unique identifier for each bank account...",
        "district_id": "Identifier representing the district...",
        ...}
    }...}
  ...
}
```

Fig. 4: Example JSON specification of the knowledge base

References

1. Ji, Z., et al.: Towards general-purpose in-context learning for knowledge extraction. arXiv preprint arXiv:2305.12253 (2023)
2. Lobo, E., et al.: Matching business glossaries to databases: A semantic and few-shot learning approach. In: Proceedings of the 2023 International Conference on Management of Data (2023)
3. Gao, Y., et al.: Automatic metadata generation for text-to-sql. To appear in: Proceedings of the VLDB Endowment (2025)
4. Qin, K., et al.: Relational schema-augmented text-to-sql generation. In: Proceedings of the ACM Web Conference (2024)
5. Wretblad, P., et al.: Synthetic data for text-to-sql: A simple and effective approach. In: Proceedings of the Conference on Empirical Methods in Natural Language Processing (2024)
6. Ma, Z., et al.: Enhancing text-to-sql with domain knowledge injection. In: Proceedings of the AAAI Conference on Artificial Intelligence (2024)
7. Lewis, P., et al.: Retrieval-augmented generation for knowledge-intensive nlp tasks. In: Advances in Neural Information Processing Systems (2020)
8. Asai, A., et al.: Self-rag: Learning to retrieve, generate, and critique through self-reflection. arXiv preprint arXiv:2310.11511 (2023)
9. Chan, H., et al.: Rq-rag: A retrieve-query-generate framework for in-context learning. arXiv preprint arXiv:2404.07598 (2024)
10. Zhou, Y., et al.: Openrag: An open-source retrieval-augmented generation framework. To appear in: Proceedings of the AAAI Conference on Artificial Intelligence (2025)
11. Zhong, V., Xiong, C., Socher, R.: Seq2sql: Generating structured queries from natural language using reinforcement learning. arXiv preprint arXiv:1709.00103 (2017)
12. Xu, X., Liu, C., Song, D.: Sqlnet: Generating structured queries from natural language without reinforcement learning. arXiv preprint arXiv:1711.04436 (2017)
13. Pourreza, M., Ghasemzadeh, H.: Din-sql: Decomposed in-context learning of text-to-sql with self-correction. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (2023)
14. Sun, R., et al.: Sqlprompt: In-context learning for text-to-sql. In: Proceedings of the Conference on Empirical Methods in Natural Language Processing (2023)
15. Li, J., et al.: BIRD: A big bench for large-scale database grounded text-to-sql. In: Advances in Neural Information Processing Systems (2023)
16. Reimers, N., Gurevych, I.: Sentence-bert: Sentence embeddings using siamese bert-networks. In: Proceedings of the Conference on Empirical Methods in Natural Language Processing (2019)
17. Zheng, L., et al.: Judging llm-as-a-judge with mt-bench and chatbot arena. In: Advances in Neural Information Processing Systems (2023)
18. Guo, D., et al.: Deepseek coder: Let the code write itself. To appear in: Proceedings of the International Conference on Learning Representations (2025)
19. Gao, D., et al.: Text-to-SQL empowered by large language models: A benchmark evaluation. Proceedings of the VLDB Endowment 17(11) (2024)
20. Wang, B., et al.: MAC-SQL: A multi-agent collaborative framework for Text-to-SQL. In: Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024) (2024)
21. Pourreza, M., et al.: CHASE-SQL: Multi-Path Reasoning and Preference Optimized Candidate Selection in Text-to-SQL. arXiv preprint arXiv:2410.01943 (2024)